Design Patterns Justification

Refactored Parking Lot Manager

This document explains the three design patterns implemented in the refactored Parking Lot Manager system:

1. **MVP (Model-View-Presenter) Pattern** - Architecture
2. **Repository Pattern** - Data Access
3. **Factory Pattern** - Object Creation

Each pattern addresses specific anti-patterns identified in the original code and improves maintainability, testability, and scalability.

Pattern 1: MVP (Model-View-Presenter)

The original system had a **God Class** that mixed UI, business logic, and data access in one monolithic `ParkingLot` class. This violated the Single Responsibility Principle and made testing impossible.

Why MVP?

MVP pattern separates the application into three distinct layers:

- **Model**: Business logic and domain entities
- **View**: User interface (passive, no logic)
- **Presenter**: Coordination between Model and View

Implementation Details

Model Layer (`models/`):

```
# vehicle.py - Domain entities
class Vehicle:
    def __init__(self, reg_num, make, model, color):
        self._reg_num = reg_num
        self._make = make
        self._model = model
        self._color = color

    @property
    def reg_num(self):
        return self._reg_num
```

View Layer (`view.py`):

```
class ParkingView:
    def __init__(self, presenter):
        self.presenter = presenter
```

```
        # UI setup only, no business logic

    def display_message(self, message):
        self.output_text.insert(tk.END, message + "\n")
```

Presenter Layer (`presenter.py`):

```
class ParkingPresenter:
    def __init__(self, repository, view):
        self.repository = repository
        self.view = view

    def park_vehicle(self, reg, make, model, color, is_ev, is_motor):
        # Coordinates between model and view
        slot = self.repository.find_empty_slot(is_ev)
        if slot is not None:
            vehicle = VehicleFactory.create_vehicle(...)
            self.repository.save(vehicle, slot, is_ev)
            self.view.display_message(f"Allocated slot: {slot}")
```

Benefits Achieved

1. **Separation of Concerns**: Each layer has one responsibility
2. **Testability**: Can test presenter without GUI
3. **Flexibility**: Can swap UI framework without changing business logic
4. **Maintainability**: Changes isolated to specific layers
5. **Reusability**: Business logic can be used in CLI, API, or GUI

Why Not MVC?

MVC allows View to observe Model directly, creating two-way dependencies. MVP's one-way flow (View → Presenter → Model) provides better separation and easier testing.

Pattern 2: Repository Pattern

The original code directly manipulated arrays (`self.slots`, `self.evSlots`) throughout the business logic, making it impossible to:

- Switch to a database
- Add caching
- Implement transactions
- Mock data for testing

Why Repository?

Repository pattern provides an abstraction layer between business logic and data storage, treating data access like an in-memory collection.

Implementation Details

Interface (repositories/interface.py):

```
from abc import ABC, abstractmethod

class IVehicleRepository(ABC):
    @abstractmethod
    def save(self, vehicle, slot, is_ev):
        pass

    @abstractmethod
    def find_by_slot(self, slot, is_ev):
        pass

    @abstractmethod
    def find_by_reg_num(self, reg_num):
        pass

    @abstractmethod
    def find_empty_slot(self, is_ev):
        pass

    @abstractmethod
    def remove(self, slot, is_ev):
        pass
```

Implementation (repositories/in_memory_repository.py):

```
class InMemoryVehicleRepository(IVehicleRepository):
    def __init__(self, capacity, ev_capacity):
        self._regular_slots = {}
        self._ev_slots = {}
        self._capacity = capacity
        self._ev_capacity = ev_capacity

    def save(self, vehicle, slot, is_ev):
        if is_ev:
            self._ev_slots[slot] = vehicle
        else:
            self._regular_slots[slot] = vehicle
        return True

    def find_empty_slot(self, is_ev):
        slots = self._ev_slots if is_ev else self._regular_slots
        capacity = self._ev_capacity if is_ev else self._capacity

        for i in range(1, capacity + 1):
            if i not in slots:
```

```
                return i
        return None
```

Benefits Achieved

1. **Abstraction**: Business logic doesn't know about storage details
2. **Flexibility**: Easy to swap in-memory for database later
3. **Testability**: Can inject mock repository for testing
4. **Consistency**: Single point for data access logic
5. **Future-proof**: Can add caching, transactions, etc.

Pattern 3: Factory Pattern

The original code had vehicle creation scattered throughout the `park()` method:

```
if (motor == 1):
    self.evSlots[slotid] = ElectricVehicle.ElectricBike(...)
else:
    self.evSlots[slotid] = ElectricVehicle.ElectricCar(...)
```

This violated DRY principle and made it hard to:

- Add new vehicle types
- Ensure consistent initialization
- Validate vehicle data

Why Factory?

Factory pattern centralizes object creation in one place, providing:

- Single point of change for new types
- Consistent validation
- Type-safe creation
- Encapsulated complexity

Implementation Details

Factory Class (`factories/vehicle_factory.py`):

```
class VehicleFactory:
    @staticmethod
    def create_vehicle(vehicle_type, reg_num, make, model, color,
is_electric=False):
        """
        Central factory for creating all vehicle types.

        Args:
            vehicle_type: 'car' or 'motorcycle'
```

```
            is_electric: True for electric vehicles

        Returns:
            Vehicle instance

        Raises:
            ValueError: Invalid vehicle type
        """
        vehicle_type = vehicle_type.lower()

        if is_electric:
            if vehicle_type == 'car':
                return ElectricCar(reg_num, make, model, color)
            elif vehicle_type == 'motorcycle':
                return ElectricBike(reg_num, make, model, color)
            else:
                raise ValueError(f"Unknown electric vehicle type: {vehicle_type}")
        else:
            if vehicle_type == 'car':
                return Car(reg_num, make, model, color)
            elif vehicle_type == 'motorcycle':
                return Motorcycle(reg_num, make, model, color)
            else:
                raise ValueError(f"Unknown vehicle type: {vehicle_type}")
```

Benefits Achieved

1. **Centralized Creation**: One place to create all vehicles
2. **Type Safety**: Validates vehicle types
3. **Easy Extension**: Add new types in one place
4. **Consistent Initialization**: All vehicles created the same way
5. **Error Handling**: Clear error messages for invalid types

---

Pattern Interactions

The three patterns work together to create a clean architecture:

```
User Interaction
      ↓
   ParkingView (MVP - View)
      ↓
   ParkingPresenter (MVP - Presenter)
      ↓                    ↓
VehicleFactory (Factory)   IVehicleRepository (Repository)
      ↓                    ↓
   Vehicle (MVP - Model)   Storage
```

Flow Example (Parking a car):

1. User clicks "Park Car" in **View**
2. View calls **Presenter**.park_vehicle()
3. Presenter asks **Repository** for empty slot
4. Presenter uses **Factory** to create Vehicle
5. Presenter tells **Repository** to save vehicle
6. Presenter updates **View** with result

Each layer is independent and testable.

Future Extensibility

The refactored design supports:

1. **Web API**: Replace View with REST controller
2. **Mobile App**: New View, same Presenter
3. **Database**: New Repository implementation
4. **Microservices**: Repository can call remote services
5. **Event Sourcing**: Repository can publish events
6. **Caching**: Decorator pattern on Repository

This can now be done without changing core business logic.